

Experimentation with Solar Flare Prediction and Temporal Information

By Gora Bepary

Motivation and Goals

With the recent Artemis II moon flyby mission, I was inspired to work with data regarding outer space. I came across some interesting projects working with solar flare prediction and also NASA's Solar Dynamics Observatory (SDO). Solar flares are bursts of radiation released from the Sun, typically associated with active regions around sunspots. They occur when magnetic energy built up in the solar atmosphere is released, accelerating particles and emitting immense energy. Predicting solar flares using images involves analyzing the sun's surface, especially X-ray imagery—from space-based telescopes. NASA has 2 such instruments: the Atmospheric Imaging Assembly (AIA) instrument and the Helioseismic and Magnetic Imager (HMI). These images reveal patterns in active regions, magnetic field complexity, and rapid changes in structure. Machine learning models are often trained on sequences of such images to detect magnetic shear, flux emergence, or irregular sunspot group evolution, all of which can signal an increased likelihood of a flare. Prediction is important because solar flares can have significant effects on Earth. They can disrupt satellites, GPS systems, and radio communications. Early warnings allow operators of critical infrastructure to take protective measures, making solar flare prediction an essential part of modern space weather forecasting.

I used this project opportunity to learn more about neural networks and image processing, as they are things that are very relevant in the modern Machine Learning space. The goal of the project is to learn and apply techniques using neural networks to derive temporal information in solar data, rather than simply get a high accuracy.

Project Repo

Github link : <https://github.com/Luminous271/Solar-Flare-Prediction.git>

training.ipynb contains my code for training the models.

At the top of this notebook **project.ipynb** automatically runs relevant models and evaluates them. I also display GRAD-CAM visualization for this. I have a couple different models saved to the repo which are not all used. Final Models used for evaluation:

Data

Website: <https://i4ds.github.io/SDOBenchmark/#dataset>

SDO Benchmark is a dataset benchmark that was in development a couple years ago but is still in use today as a part of different projects. I downloaded the data straight from the website, and also from the relevant Kaggle page. What makes this dataset hard is a couple things:

Sample complexity, sampleset size, and regression with heavy-tailed targets. Each sample consists of only 4 images within a 12 hour period before a 24 hour prediction period. These images are very close together in time, and also with their being 4 of them it makes it difficult to learn valuable information. Additionally, there are very few samples because this is a heavily processed subset of raw SDO data. In total there are around 7000 plus samples, which 40 images each - 10 imaging types for each timeframe. The reason for so few samples has to do with not being able to simulate solar flares in addition to a strategic sample choice. If we had a ton of images for only a few active regions (ARs), models would simply learn to look up whether specific ARs or patches on the image would flare. This is also a reason why they prioritized getting different active regions and not random patches or the same patches. The data is labeled via peak_flux emission, manually by experts. Solar flares are themselves rare so there is extreme class imbalance in the labeling, so MAE as an evaluation metric is used. Peak_flux itself is logarithmic in nature (heavy-tailed) with rare high values, and common lower values. There are some other problems regarding the samples and solar flare prediction in general like sensor degradation and also the data not being stable because of a 11-year solar cycle (the magnetic poles of the sun flip). In general I ignored these problems for my experimentation. The provided data analysis from where most of my information came from:

https://github.com/i4Ds/SDOBenchmark/blob/master/notebooks/data_analysis.ipynb.

Additionally, when predicting all flares with the same fixed peak_flux, we get the following MAE - train: 3.3363243655142895e-06 test: 1.5358676404195967e-05. This is also given by the data analysis.

Models and Training Regime

I used Pytorch to create my models, matplotlib for visualizations, and also torch for Grad-CAM visualizations. Initially, I was quite naive in my approach to model formulation, and did not really look in depth into existing projects before starting. For the presentation, I only used around ¼ of the dataset, which resulted in very bad/inconsistent results. I proceeded as follows, using HMI magnetogram images and the full dataset for training and testing 3 different models: BaselineCNN:

The model takes a single-channel 2D solar magnetogram image as input. It consists of two convolutional blocks, increasing the channel depth from $1 \rightarrow 16 \rightarrow 32$. Each convolution is followed by a ReLU activation and 2×2 max pooling, progressively reducing spatial resolution from $224 \times 224 \rightarrow 112 \times 112 \rightarrow 56 \times 56$. The resulting feature maps are flattened and passed through a fully connected layer with 64 hidden units. A final linear layer maps the 64-dimensional representation to a single scalar output, corresponding to the predicted peak flux value in a regression setting.

Intuition: Simple/shallow CNN. Used as an encoder for the next few models.

Training Regime: L1-Loss (MAE), Patience (how many bad epochs to tolerate) of 4 and a learning rate of 1e-3. The model had an early stop at around epoch 6.

Testing Results: Looks like it generalizes pretty well, test loss is very similar to validation loss and training loss. As we can see, validation loss is pretty unstable.

Final Test Loss: 0.00012316565866617917

Temporal Attention:

Each of the 4 temporal images is passed through a shared CNN encoder (reused the one from before) to produce a feature vector, where each frame is represented by learned spatial features. A small attention layer assigns a learned importance weight to each frame, using a softmax so weights sum to 1 across time, allowing the model to focus on the most informative moments in the 12-hour window. The per-frame features are combined via a weighted sum, producing a single fused representation. Regression head for prediction: The fused feature vector is passed through a small MLP to output a single scalar (peak flux), while also returning the attention weights for interpretability (i.e., which frames mattered most).

Intuition: Focus on the most informative time step, Learn when changes matter.

Training Regime: I lost the resulting loss values for each epoch, but it trained for the full 15 epochs and I believe the best epoch was around 9-11 when it started to fail the patience test (validation loss started increasing consistently).

Testing Results: Showed similar results to the best epochs validation and training loss.

Final Test Loss: 0.000039

Temporal Frame Difference:

Frame-wise encoding: Each of the 4 input frames is passed through a baseline CNN encoder to produce feature vectors. Temporal difference modeling: The model explicitly computes feature differences between consecutive frames (d1, d2, d3), capturing how the solar scene evolves over time rather than just its static appearance. Feature fusion: It concatenates the final frame features (f4) with the temporal differences, combining current state + recent changes into a single representation [f4, d1, d2, d3]. Regression prediction: The combined feature vector is passed through a small MLP to output a single scalar (peak flux), leveraging both spatial features and temporal evolution.

Intuition: Captures temporal change explicitly. Uses evolution, not just state. Here we use feature engineering to create new features.

Training Regime: Same parameters as before (learning rate, weight decay).
Validation loss was slightly unstable, it looks like the best epoch was epoch 5.

```
Epoch 1/15 | Train: 0.000457 | Val: 0.000490  
Epoch 2/15 | Train: 0.000240 | Val: 0.000075  
Epoch 3/15 | Train: 0.000210 | Val: 0.000123  
Epoch 4/15 | Train: 0.000227 | Val: 0.000245  
Epoch 5/15 | Train: 0.000240 | Val: 0.000051  
Epoch 6/15 | Train: 0.000213 | Val: 0.000097  
Epoch 7/15 | Train: 0.000167 | Val: 0.000289  
Epoch 8/15 | Train: 0.000190 | Val: 0.000171  
Epoch 9/15 | Train: 0.000091 | Val: 0.000156  
Early stopping triggered
```

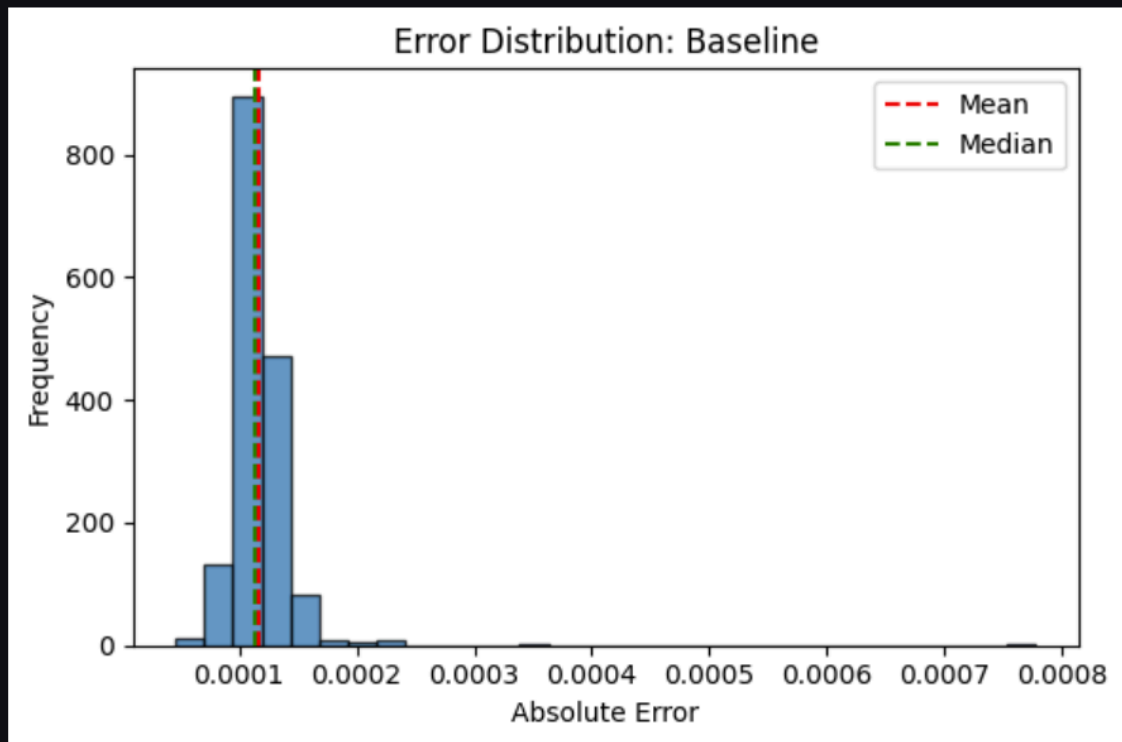
Testing Results: Reasonably close to the best validation loss.

```
Final Test Loss: 0.000061
```

Results

===== Baseline Error Summary =====

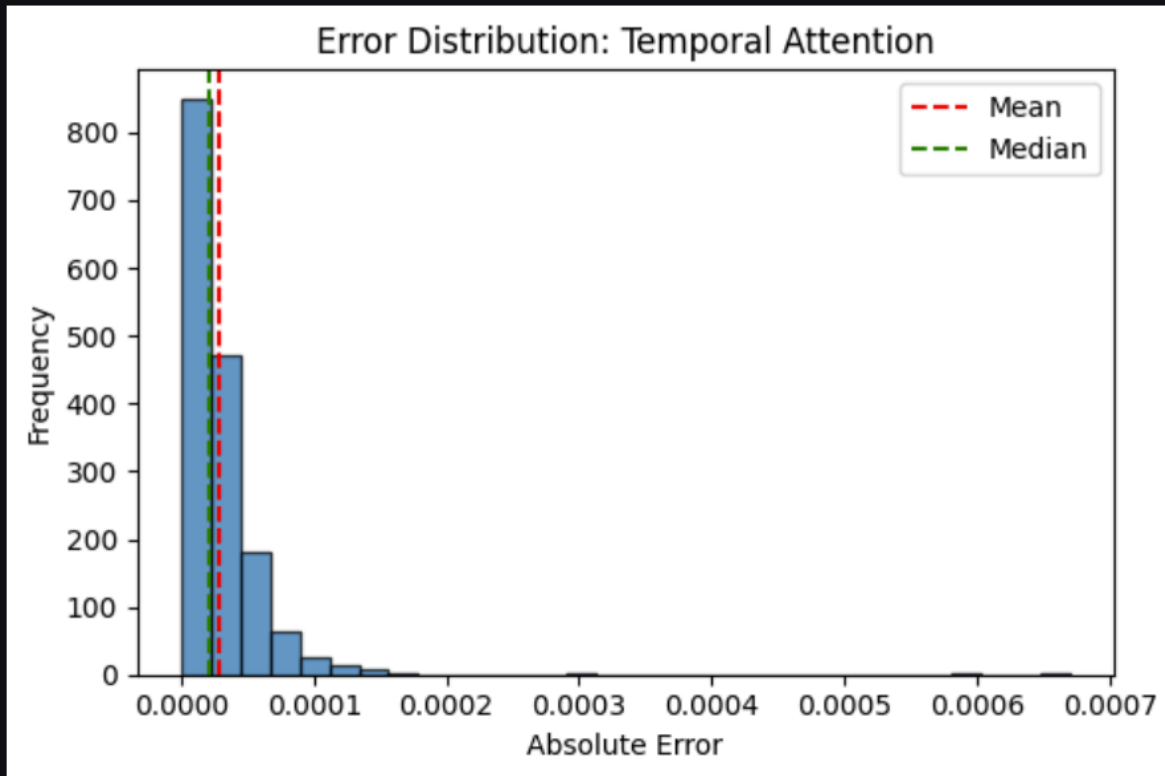
Mean : 1.168349e-04
Std : 3.076717e-05
Median : 1.139759e-04
90th pct : 1.382775e-04
95th pct : 1.474737e-04
99th pct : 1.959043e-04
Min : 4.622544e-05
Max : 7.771553e-04



Remember this model is trained on a single frame from each sample (the first). Compared to the other two models, BaselineCNN had higher MAE overall. However, it is more stable in comparison (lower std).

===== Temporal Attention Error Summary =====

Mean : 2.887211e-05
Std : 3.346868e-05
Median : 2.137714e-05
90th pct : 6.031940e-05
95th pct : 7.645789e-05
99th pct : 1.188207e-04
Min : 4.430650e-08
Max : 6.698825e-04

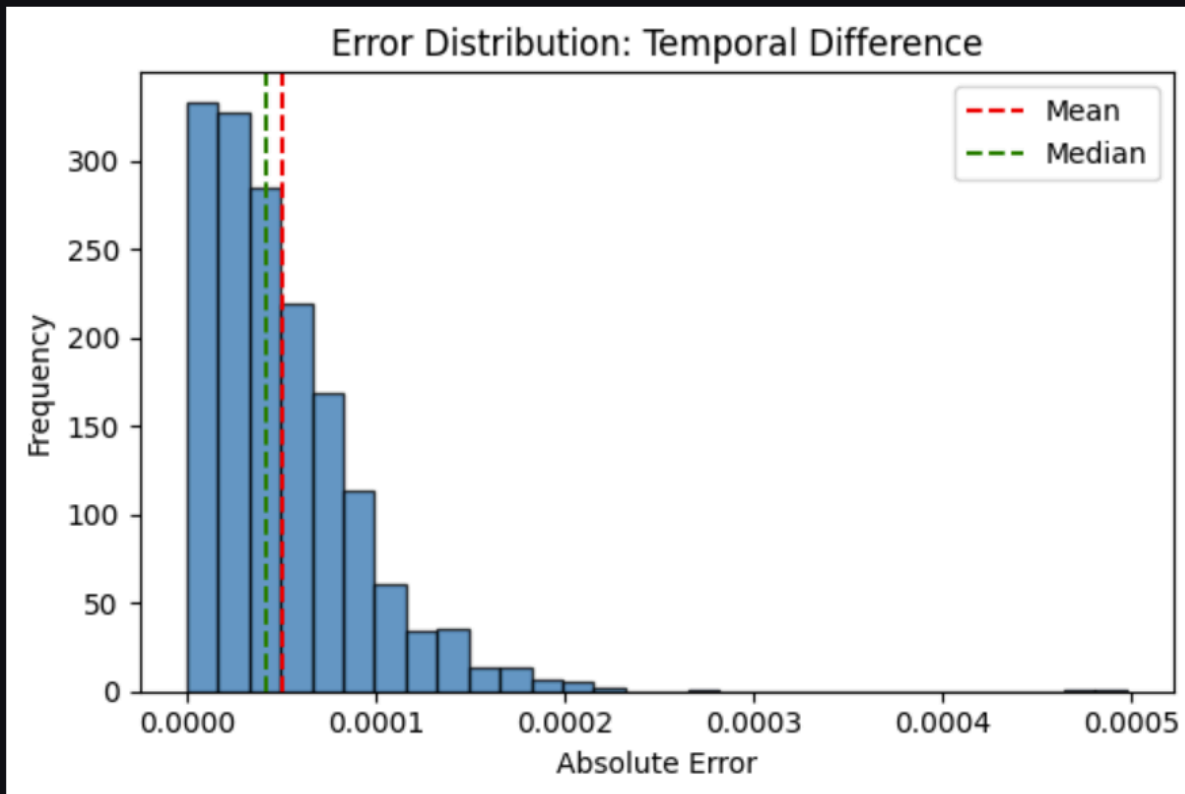


The temporal attention model had the best mean and median. There is also a better tail performance (low 90/95/99 percentile error). It benefitted heavily from more samples. I trained the attention model and the frame difference model on only 1000 samples and they did way worse than the baseline model. When I increased complexity, it was able to extract some more meaningful information from the 4 frames.

However, there is a caveat: when printing out the final attention weights, they appear to be really close to each other. This makes sense because the 4 frames were very close together, but it doesn't really prove that this model was learning an important temporal signal. It was simply acting closer to mean-pooling or averaging the 4 frames together.

===== Temporal Difference Error Summary =====

Mean : 5.061758e-05
Std : 4.250030e-05
Median : 4.225497e-05
90th pct : 1.021008e-04
95th pct : 1.314910e-04
99th pct : 1.850992e-04
Min : 8.970147e-08
Max : 4.973243e-04

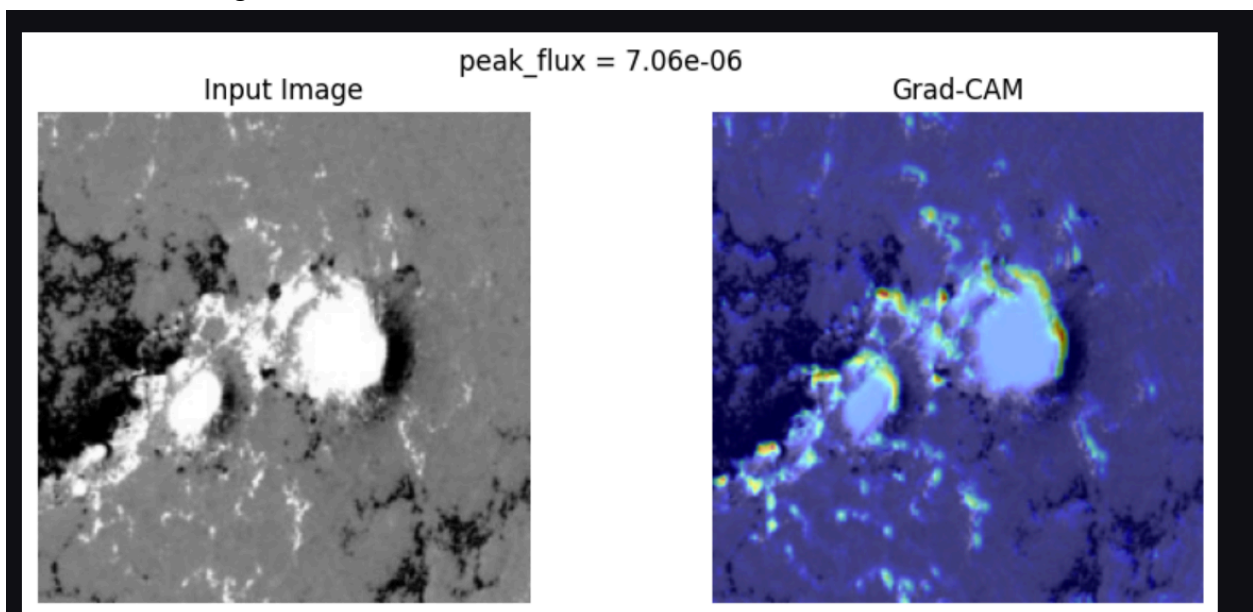


The last model used some feature engineering to produce differences between the 4 frames along with the last frame. This is around 2x better than baseline, it shows that meaningful information was captured from handcrafted differences. However, this model is less consistent, and has a higher std than BaselineCNN.

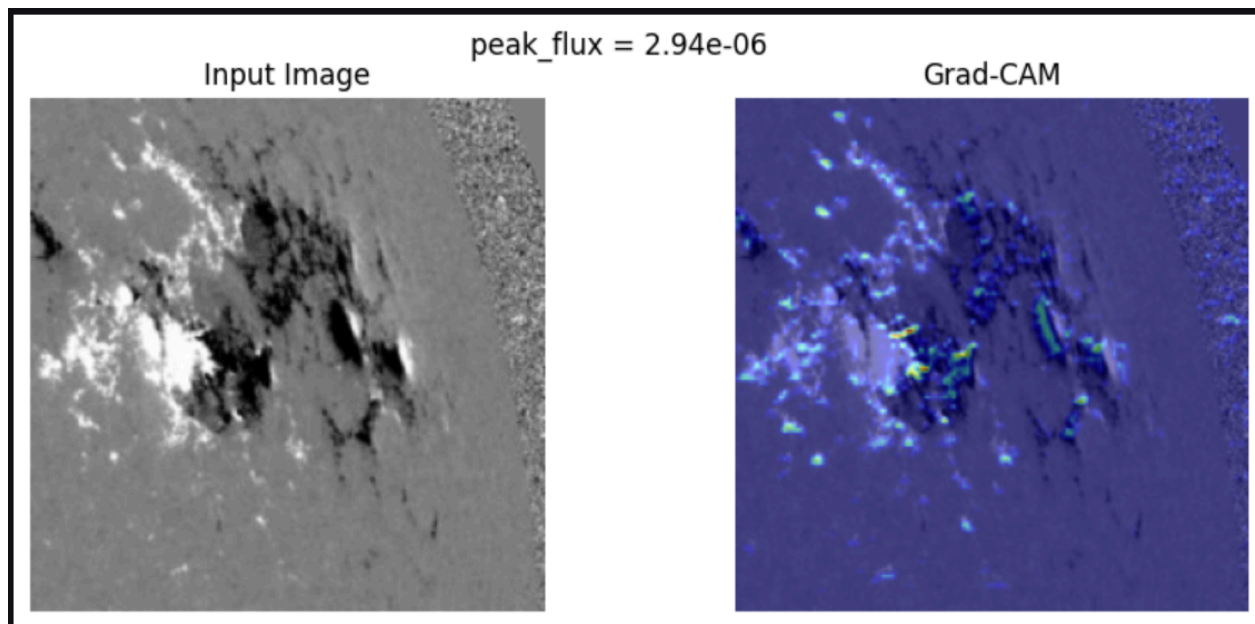
In general all three of my models do a little worse than the fixed peak_flux predictions. I think if I refine my training even more, with reduced learning rates and fine tuned parameters I can get an even better MAE for all three. The creator of this benchmark managed to get an MAE of way better than guessing a fixed peak_flux for all samples. However, my original goal was to see if the information in a small time frame of 12 hours could be utilized rather than just using 1 frame, and which temporal formulations work better. I would say this project was generally a success because of this. However, the fact the attention model converged to averaging the weights together shows that simply averaging the weights did better than both. I would label this project as a partial success, open to more work and refinement but a good learning experience.

Visualizations

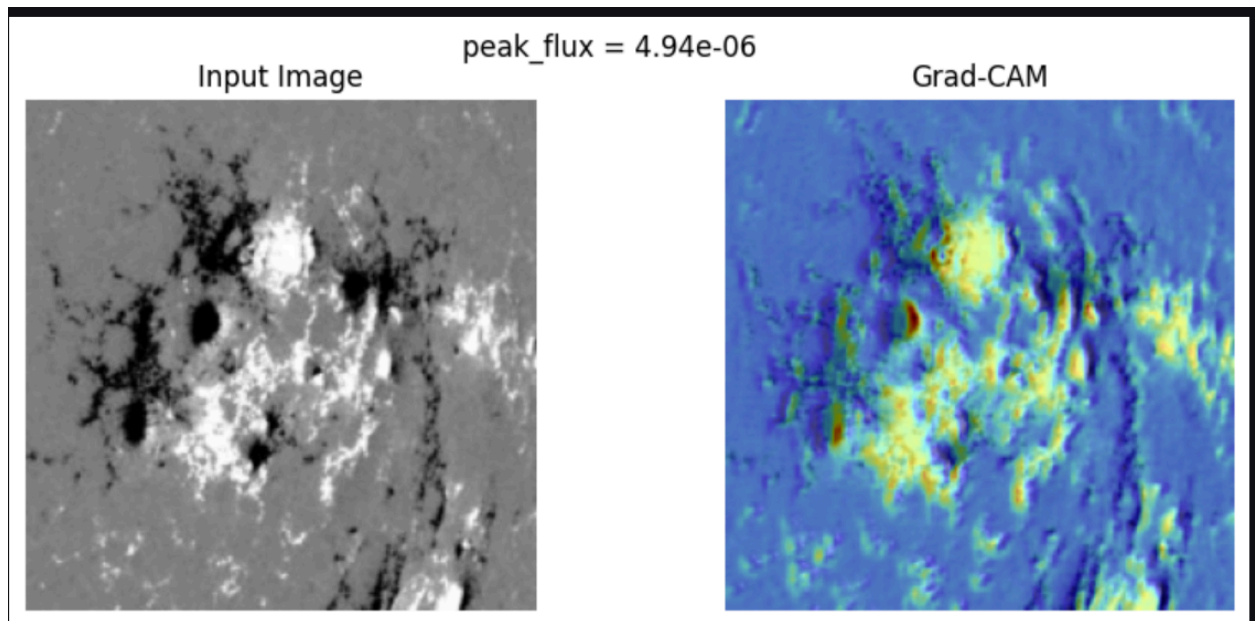
I did some grad-cam visualizations for BaselineCNN. These are some notable ones:



It looks like the model is taking information from the boundary of the white and black areas, which are opposite in polarity. It shows that some sort of underlying physics is informing the model. I did some reading and areas with strong magnetic shear. However, this is not the pattern observed for all samples and it doesn't necessarily mean that predictions will be accurate. It is simply interesting to look at, and might indicate that the model is heading in the right direction.



Another interesting visualization. You can see some of the boundary highlighting patterns I mentioned earlier, but also highlighting around the more “textured” areas.



Again there is red highlighting near boundaries between white and black. However we see that the surrounding hue is lighter.

These visualizations would be more helpful if I attached prediction values to them, which would aid us in seeing what areas are useful in accuracy, but I would argue it is a little hard to understand now because base accuracy is still way worse than predicting a fixed flux value for all samples. I could improve interpretation of these images if I had closer accuracy, but for now it is interesting to look at the parts of the image used for prediction.

Background & Related Work

Some of these are articles and projects I skimmed through and read for inspiration. Others are just ones I found interesting regarding this subject. A couple articles I had to refer to a lot for my implementation.

- The dataset loader and the BaseCNN data structures are inspired by this kaggle submission using pytorch. The other submissions on there were quite helpful in figuring out how to start with this project.:

<https://www.kaggle.com/code/ivantang86/flare-prediction-bycnn>

This kaggle submission had some interesting Grad-CAM visualizations:

<https://www.kaggle.com/code/brsdincer/solar-flares-prediction-unsupervised-learning>

This article was very useful when implementing my Grad-CAM structure and understanding how it works:

<https://medium.com/@stepanulyanin/implementing-grad-cam-in-pytorch-ea0937c31e82>

- Regarding temporal attention - There were numerous articles I read about the theoretical and technical applications of temporal attention. The wikipedia page is where I started:

https://en.wikipedia.org/wiki/Visual_temporal_attention

<https://www.frontiersin.org/journals/cognition/articles/10.3389/fcogn.2023.1168320/full#s2>

- Solar Flare Prediction in General:

I liked this paper a lot when reading up on the context of solar flare prediction. It highlights how high-energy solar flares and coronal mass ejections pose a major threat to Earth's infrastructure, with the potential for widespread economic and societal disruption. The paper proposes a satellite-based, edge-computing system using onboard machine learning to predict solar events in real time and enable early mitigation measures. There are a lot of interesting things explained in this paper which I appreciated.

<https://commons.erau.edu/cgi/viewcontent.cgi?article=1218&context=db-srs>

NASA's SDO mission resources:

<https://data.nasa.gov/dataset/solar-dynamics-observatory>

Discussion and Closure

I have learned a lot from this class and appreciate machine learning as a whole a lot more than I used to. This project is one of my first full machine learning projects at a larger scientific scale, so I struggled initially. However, applying knowledge on something practical like this was actually really enjoyable, working with scientific data like Solar Images helped me realize a new love for computer science applied in astronomy. One point I learned was the importance of formulating a model properly with training regime and consulting an analysis of the data before starting model training. Training is an expensive thing, so not having a good training regime and an intuition of how well the model will generalize based on its and the data's properties will waste a lot of time. I think my overall accuracy could be improved at least 2x if I refined my training methods and model formulations. My experiments might also have inconsistencies that can alter my final conclusions, so I would say in general there is a lot of room for refinement. While there was not a ton of empirical success in my endeavors I was able to learn valuable techniques with techniques we learned in class.